E-Commerce Website Project

Project Specification: Full-Stack E-Commerce Website

General Overview

You are required to develop a complete e-commerce website with two distinct interfaces:

- **Customer front-end (for browsing, cart, orders, profile)**
- **Admin back-end (for managing products, orders, users, messages) The system must be built using:**
- **Front-end: HTML5, CSS3, (JavaScript if needed) (responsive design, no external libraries required unless desired).**
- **Back-end: PHP for server-side logic.**
- **Database: MySQL with the seven tables specified below.**

All data must be stored in the database. No hard-coded content. The front-end pages must communicate with PHP scripts via form submissions (POST/GET) for dynamic updates (e.g., add to cart without page reload – recommended but not mandatory).

Part 1: Database Schema (7 Tables)

Create a MySQL database named ecommerce_db with the following tables. Define appropriate data types, primary keys, foreign keys, and indexes.

1. users

Stores both customers and admins.

Column	Type	Constraints
user_id	INT	PRIMARY KEY, AUTO_INCREMENT
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL (store hashed password)
phone	VARCHAR(20)	
address	TEXT	
role	ENUM('customer','admin')	DEFAULT 'customer'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

2. categories

Column	Type	Constraints
category_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(50)	UNIQUE, NOT NULL
description	TEXT	

3. products

Column	Type	Constraints
product_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(150)	NOT NULL
description	TEXT	
price	DECIMAL(10,2)	NOT NULL
stock_quantity	INT	DEFAULT 0
image_url	VARCHAR(255)	(path or URL to product image)
category_id	INT	FOREIGN KEY → categories(category_id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4. cart_items

Stores items in a user's shopping cart before checkout.

Column	Type	Constraints
cart_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY → users(user_id) ON DELETE CASCADE
product_id	INT	FOREIGN KEY → products(product_id) ON DELETE CASCADE
quantity	INT	NOT NULL, CHECK (quantity > 0)
added_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
UNIQUE(user_id, product_id) – each product appears once per user in cart.		

5. orders

Column	Type	Constraints
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY → users(user_id) ON DELETE RESTRICT
order_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
total_amount	DECIMAL(10,2)	NOT NULL
status	ENUM('pending','paid','shipped','delivered','cancelled')	DEFAULT 'pending'
shipping_address	TEXT	NOT NULL (copied from user's address at checkout)

6. order_items

Column	Type	Constraints
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FOREIGN KEY → orders(order_id) ON DELETE CASCADE
product_id	INT	FOREIGN KEY → products(product_id)
quantity	INT	NOT NULL
unit_price	DECIMAL(10,2)	NOT NULL (price at time of purchase)

7. contacts

Column	Type	Constraints
message_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL
subject	VARCHAR(100)	
message	TEXT	NOT NULL
is_read	BOOLEAN	DEFAULT FALSE
submitted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Part 2: Front-End Pages – Design Requirements

Students must design the following HTML/CSS/JavaScript pages. Each page must have a consistent layout (header, navigation, footer, responsive grid/flex). The navigation menu should change based on login status and user role.

A. Public & Customer Pages (8 pages)

1. index.html (Home Page)

- **Hero section with a welcome message and call-to-action button (link to products).**
- **Featured products section: display 4-6 products fetched from the database (via PHP include or AJAX). Each product shows image, name, price, and an "Add to Cart" button.**
- **Search bar: allows searching products by name (submits to products.html with a query parameter).**
- **Navigation links: Home, Products, Cart, Contact. If user not logged in, show "Login/Register". If logged in, show "My Profile" and "Logout".**

2. products.html (or products.php – all pages can be .php for dynamic content)

- **Display all products in a responsive grid (image, name, price, category).**
- **Filtering: dropdown or buttons to filter by category (values from categories table).**
- **Sorting: sort by price (low to high, high to low).**
- **Each product card includes "View Details" button (links to product-detail.html?id=...) and "Add to Cart" button.**

3. product-detail.html (Dynamic page – use query string ?id=...)

- **Fetch and display full product information (name, description, price, stock, image).**
- **Quantity selector (number input, min=1, max=stock).**
- **"Add to Cart" button – adds selected quantity to cart (for logged-in users, updates cart_items table via PHP; for guests, can use JavaScript localStorage but specification requires eventual database storage after login).**
- **"Back to Products" link.**

4. cart.html (Shopping Cart Page)

- **Display cart items for the logged-in user (fetched from cart_items joined with products).**
- **Show: product image, name, unit price, quantity (editable), subtotal, remove button.**
- **Update quantity – user changes quantity, page sends AJAX or form POST to update cart_items table.**
- **Remove item – delete from cart_items.**
- **Cart summary (subtotal, tax, total).**
- **"Proceed to Checkout" button. If user not logged in, redirect to login.html. After login, return to cart.**
- **Checkout process: When user clicks "Place Order", create a new record in orders and copy cart items to order_items, then clear cart_items for that user.**

5. contact.html

- **Form with fields: Name, Email, Subject (dropdown: Inquiry / Complaint / Suggestion), Message (textarea).**
- **Submit – sends data to a PHP script (contact_process.php) that inserts the message into the contacts table and shows a success message.**
- **Also display static contact information (address, phone, email, working hours).**

6. login.html

- **Form: email, password.**
- **On submit, send to login.php. Validate credentials against users table. If correct, start a session and store user_id, full_name, role. Redirect: if role = 'admin' → admin_dashboard.php; else → index.html.**
- **Show error message for invalid login.**

7. register.html

- **Form: full name, email, password, confirm password, phone, address.**
- **On submit, send to register.php. Check if email already exists. If not, hash password and insert new user with role = 'customer'. Redirect to login.html.**

8. profile.html (or profile.php)

- **Accessible only to logged-in customers.**
- **Display user information (name, email, phone, address) with an edit form to update.**
- **Order History: list all orders for this user (from orders table) showing order ID, date, total, status. Each order has a link to view order details (items purchased from order_items).**
- **Logout button (link to logout.php that destroys session).**

B. Admin Pages (5 pages – accessible only to role = 'admin')

All admin pages must check session role at the top; if not admin, redirect to index.html.

1. admin_dashboard.php

- **Summary cards: total products (count from products), total orders, total users, unread contact messages (count where is_read = false).**
- **Quick links to the management pages below.**

2. admin_products.php

- **List products in an HTML table with columns: ID, name, price, stock, category, edit button, delete button.**
- **Form to add new product: inputs for name, description, price, stock, image URL, category (dropdown from categories table).**
- **Edit product: when clicking "Edit", populate a form with existing data; update the product in database.**
- **Delete product: with confirmation (remove from products table; optionally handle foreign keys in cart_items and order_items with ON DELETE SET NULL or restrict).**

3. admin_orders.php

- **List all orders (join with users to show customer name). Columns: order ID, customer name, date, total amount, status (dropdown to change).**
- **Change status – update status column in orders table.**
- **View details – show order items (product name, quantity, unit price) for that order.**

4. admin_users.php

- **List all users (ID, name, email, role). For each user, a dropdown to change role (customer/admin) and a "Delete" button.**
- **Delete user: only allowed if user has no orders (check orders table). Show warning otherwise.**

5. admin_messages.php

- **List messages from contacts table. Show name, email, subject, message, date, and a column indicating read/unread.**
- **Mark as read button – set is_read = true.**
- **Delete message button.**

Part 3: Back-End Functional Requirements (PHP & MySQL Integration)

The front-end pages (HTML) is templates. They must be converted to .php files where dynamic content is injected by PHP. Students must write the following PHP scripts (or handle logic within the same file using includes).

1. **Database Connection**
   - **Create config.php that establishes a MySQLi or PDO connection to the database. Include this file in all PHP pages.**
2. **Session Management**
   - **Start session_start() on all pages that require login.**
   - **Store $_SESSION['user_id'], $_SESSION['role'], $_SESSION['full_name'] after successful login.**
   - **Create logout.php to destroy session and redirect to index.html.**
3. **Core PHP Scripts (by function)**

Authentication

- **login.php: receive email/password, verify against users table (compare hashed password using password_verify()), set session, redirect.**
- **register.php: insert new user after hashing password.**
- **logout.php: destroy session.**

Product Display

- **index.php and products.php: fetch products from products join categories to display. Use $_GET for filtering (category_id) and sorting.**
- **product_detail.php: fetch single product by $_GET['id'].**

Cart Operations (for logged-in users)

- **add_to_cart.php: receive product_id and quantity (via POST or GET). Insert or update cart_items for the current user.**
- **update_cart.php: receive cart_id or product_id and new quantity, update cart_items.**
- **remove_from_cart.php: delete row from cart_items.**
- **cart.php: query cart_items + products to display cart.**

Checkout

- **checkout.php: when user confirms order:**
  - **Calculate total from cart items.**
  - **Insert into orders (user_id, total_amount, shipping_address taken from user's address).**
  - **Insert each cart item into order_items (product_id, quantity, unit_price).**
  - **Delete all cart items for this user.**
  - **Redirect to order confirmation page.**

User Profile

- **profile.php: display user info; provide form to update users table (update name, phone, address). Handle password change separately.**
- **order_history.php: fetch orders and display.**

Admin Scripts

- **Each admin page (e.g., admin_products.php) must include session_start() and check $_SESSION['role'] == 'admin'.**
- **Perform CRUD operations using SQL queries.**
- **For product add/edit: use INSERT and UPDATE.**
- **For order status change: UPDATE orders SET status = ….**
- **For user role change: UPDATE users SET role = ….**
- **For deleting product: DELETE FROM products WHERE product_id = ... (consider constraints – set category_id to NULL or handle).**

Contact Form

- **contact_process.php: insert data into contacts table; redirect back to contact.html with success message.**

Part 4: Functional Flow & Integration (What the Student Must Ensure)

1. **Guest user can browse products and add items to cart (cart stored in cart_items table only if logged in – so either implement guest cart with sessions/localStorage, or require login before adding. Specification recommends: allow guests to add to cart using JavaScript localStorage, then upon login, merge localStorage cart into database cart_items. Students must choose and implement one method).**
2. **Registered customer can log in, view profile, edit profile, see order history, place orders, and manage cart (database-backed).**
3. **Admin can log in and access all admin pages, manage products, update order statuses, change user roles, view/delete contact messages.**

4. **All data displayed on front-end pages (products, categories, cart, orders) must come from the MySQL database via PHP. No static HTML content except layout.**
5. **Security (conceptual):**
    - **Use htmlspecialchars() when outputting data to prevent XSS.**
    - **Use prepared statements (PDO or MySQLi) to prevent SQL injection.**
    - **Hash passwords with password_hash().**
    - **Validate all inputs on server side.**

Part 5 Deliverables (for Students)

1. **Complete front-end code** (HTML5, CSS3 including in PHP) for all pages, with responsive design.
2. **Database schema** (SQL script) to create all tables with primary/foreign keys.
3. **Back-end integration** (using PHP server-side language) that:
    - Connects to the database.
    - Provides API endpoints for product listing, cart operations, and contact form submission.
    - Implements user authentication (registration/login).
4. **Project report** (PDF) containing:
    - Screenshots of each page.
    - Explanation of how the front-end communicates with the database.
5. **One Zipped File:** Upon completion, the student must submit a compressed archive (ZIP format) for all the above files. The zipped file must be in your name: [Your Firstname]_[Your Lastname].zip (e.g., Moamar_Elyazgi.zip).